Guide de recréation — Livre de recettes

Spécification complète pour recréer ou faire évoluer l'application · Nicolas BUQUET

Document de spécification destiné à une intelligence artificielle (ou à un développeur) qui devrait recréer, maintenir ou faire évoluer l'application web « Mon livre de recettes ». Lire ce guide **et** le dépôt source avant toute modification.

Propriétaire : Nicolas BUQUET (compte GitHub `nicobuq8-hue`)
Site publié : https://nicobuq8-hue.github.io/livre-recettes/
Dépôt : https://github.com/nicobuq8-hue/livre-recettes
Langue de l'interface et des échanges avec l'utilisateur : le français.
Fuseau horaire de l'utilisateur : Europe/Paris.

Résumé exécutif

Le livre de recettes est une **application web progressive (PWA)** statique, sans serveur métier. Elle s'installe sur iPhone (Safari → Partager → Sur l'écran d'accueil), reste lisible hors ligne, et recalcule les quantités d'une fiche quand on change le nombre de parts, de pâtons ou de grammes.

Elle est écrite en **HTML, CSS et JavaScript vanilla** (pas de framework, pas de backend). L'hébergement est **GitHub Pages**, branche `main`. Un service worker généré par `build-sw.py` interroge **d'abord le réseau**, puis se rabat sur le cache : les mises à jour (photos, nouvelles recettes) apparaissent sans attendre une invalidation manuelle.

Les recettes « intégrées » vivent dans `js/recettes.js`. Celles ajoutées ou modifiées depuis le téléphone sont enregistrées dans `data/recettes-ajoutees.json` via l'API GitHub Contents, grâce à un **jeton fine-grained** stocké uniquement dans le `localStorage` de l'appareil (jamais dans le dépôt). Une recette intégrée « supprimée » n'est pas effacée du JS : son `id` est ajouté à `data/recettes-masquees.json`. Au chargement, l'application fusionne les trois sources.

Règles du propriétaire à ne jamais enfreindre :

- Un tableur Excel n'est une source **que pour les quantités**. Les étapes et le ton des instructions viennent de l'ancienne fiche.
- Ne jamais écrire un poids fixe dans un libellé (`l` ou un texte `t` sans quantité) : il ne suivrait pas le réglage (bug de la pizza, « toute la biga »).
- La pâte à pizza sépare **biga** et **pâte** ; elle n'a pas d'étape de cuisson au four ; la tuile Cuisson dit « Au moment de faire la pizza » ; le sel et l'huile sont hors du poids du pâton (environ 275 g).
- Le boudin blanc part sur **1 000 g de préparation** ; les calculs se font toujours à partir de la **masse de viande** (`masseViande: 1.98`) ; le boyau est proportionnel (15 cm par boudin de 125 g).
- Photos réalistes, style photo ; le pâté doit rester rustique.
- Interface et messages en français. Confirmer avant toute suppression.
- Pas de bandeau jaune expliquant la multiplication des quantités. Le bouton « Revenir à la recette d'origine » reste.

Pour travailler : lire ce document et le dépôt ; modifier `js/recettes.js` **ou** le JSON d'ajout, pas les deux pour la même fiche sauf intégration définitive ; lancer `python3 build-sw.py` après chaque changement de fichiers cachés ; `node --check` sur les JS ; ne jamais committer de jeton ; pousser sur `main` (réessayer en cas d'erreur 500 GitHub) ; parler français à l'utilisateur.

Table des matières

1. But du projet
2. Stack technique
3. Architecture des fichiers
4. Format exact d'une recette
5. Règles métier du propriétaire
6. Calculateur de quantités
7. Ajout, modification et suppression depuis le téléphone
8. Import depuis une IA
9. Déploiement
10. Comment une IA doit travailler
11. Liens et chemins utiles
12. Annexes

---

1. But du projet

Créer un **livre de recettes personnel, clair, installable sur iPhone**, que le propriétaire peut enrichir **sans passer par un bot** :

- consulter les fiches hors ligne, avec photo, infos (parts, préparation, cuisson, température), ingrédients cochables et étapes numérotées ;
- **recalculer les quantités** avec un stepper (+ / −) et, le cas échéant, un menu de formats (ex. gros buns / mini buns) ;
- **ajouter, modifier ou supprimer** une recette depuis l'iPhone, via un formulaire qui écrit dans GitHub ;
- **importer** le contenu d'une fiche (magazine, manuscrit) en collant la réponse d'une IA tierce, sans envoi automatique ;
- imprimer ou exporter une fiche en PDF (styles d'impression dans `css/style.css`) ;
- partager une fiche (`navigator.share`) ou un lien profond du type `#/recette/buns?format=mini&n=20`.

Le livre doit rester simple à relire et à modifier plus tard par n'importe quelle IA : code vanilla, données déclaratives, commentaires en tête de `js/recettes.js`.

---

2. Stack technique

Élément	Choix
Front	HTML5 + CSS + JavaScript ES5-compatible, sans bundler ni framework
Application	SPA à une seule page (`index.html`), routage par **hash** (`#/` , `#/recette/…` , `#/ajouter` , `#/modifier/…`)
Données intégrées	`window.RECETTES` et `window.CATEGORIES` dans `js/recettes.js`
Données dynamiques	JSON statiques lus au chargement + écrits via l'API GitHub Contents
Hors ligne	Service worker `sw.js` (généré), stratégie **network-first** puis cache
PWA	`manifest.json` , icônes, meta Apple (`apple-mobile-web-app-capable` , barre translucide)
Hébergement	GitHub Pages, dépôt `nicobuq8-hue/livre-recettes` , branche `main` , site servi à la racine
Backend	**Aucun.** Pas de base de données, pas d'authentification côté app (seul le jeton GitHub côté client)
Tests minimaux	`node --check js/app.js js/ajout.js js/recettes.js` ; serveur local `python3 -m http.server 8000`

Service worker (important) : une première version servait le cache d'abord ; les photos et recettes ne se mettaient pas à jour. Il est passé en **réseau d'abord** (`fetch` avec `cache: "no-cache"` , copie en cache si OK, sinon `caches.match`). Ne pas revenir au cache-first.

Scripts, dans cet ordre (voir `index.html`) :

1. `js/recettes.js` — pousse les recettes intégrées dans `window.RECETTES`
2. `js/ajout.js` — expose `window.Ajout` (formulaire, GitHub, import IA)
3. `js/app.js` — routeur, accueil, fiche, calculateur, fusion JSON, enregistrement du SW

---

3. Architecture des fichiers

```
livre-recettes/
├── index.html                Page unique, barre du haut, <main id="app">
├── manifest.json             Nom, thème #1b5a7a, fond #f4f8fb, icônes, display standalone
├── sw.js                     Généré — ne pas éditer à la main
├── build-sw.py               Régénère sw.js (liste des fichiers + empreinte de version)
├── LISEZMOI.md               Mode d'emploi humain (structure, jeton, import IA, tests)
├── docs/                     Guides destinés aux IA (ce document + PDF)
├── css/style.css             Design mobile-first + impression
├── js/recettes.js            Recettes intégrées + documentation du format
├── js/app.js                 Affichage, recherche, calcul, fusion, routeur
├── js/ajout.js               Formulaire, jeton, API GitHub, import IA, suppression
├── data/recettes-ajoutees.json    Liste JSON des ajouts et surcharges
├── data/recettes-masquees.json    Liste JSON d'ids masqués (tombstones)
├── img/                      Photos des fiches (16:9 conseillé, ~960 px de large)
│   └── ajout-<id>.jpg        Photos envoyées depuis le formulaire (JPEG, max 1200 px)
├── icons/                    Iconographie PWA (livre de recettes physique photographié)
│   ├── favicon-32.png
│   ├── apple-touch-icon.png
│   ├── icon-192.png
│   ├── icon-512.png
│   └── icon-maskable-512.png
└── screenshots/              Captures de recette (non servies comme données)
```

Rôle de chaque pièce clé

- **`index.html`** : `lang="fr"`, viewport iPhone avec `viewport-fit=cover`, thème `#1b5a7a`, titre « Mon livre de recettes ». Le contenu est injecté dans `#app`. Le bouton retour `#btnBack` ramène à l'accueil, ou à la fiche si l'on vient de `#/modifier/<id>`.
- **`js/app.js`** : accueil (recherche accent-insensible, pastilles de catégories, cartes triées par titre `fr`), page recette (tuiles, stepper, ingrédients cochables, étapes, partager, imprimer, modifier, supprimer), fusion des JSON, deep links.
- **`js/ajout.js`** : tout ce qui écrit dans GitHub et le formulaire d'édition.
- **`css/style.css`** : jetons de couleur (voir ci-dessous), cartes 16:9, hero 4:3 (16:9 dès 560 px), stepper ambre, tuiles sarcelle, barre fixe « Retour aux recettes », styles `@media print`.
- **`build-sw.py`** : parcourt `css/`, `js/`, `icons/`, `img/`, `data/` et calcule un SHA-1 court pour le nom de cache `livre-recettes-<10 hex>`. À relancer après **chaque** ajout de recette, photo ou fichier listé. Le dossier `docs/` n'est **pas** mis en cache (volontairement).

Jetons CSS (`:root` dans `style.css`) :

Variable	Valeur	Usage
`--teal`	`#1b5a7a`	Barre du haut, boutons primaires, thème PWA
`--teal-dark`	`#134761`	Titres
`--teal-light` / `--teal-tile`	`#e6f1f7` / `#e3eef5`	Fonds de tuiles, recherche
`--amber` / `--amber-dark` / `--amber-light`	`#e8962e` / `#c97a14` / `#fdf1e0`	Stepper, pastilles, groupes
`--ink` / `--muted` / `--line` / `--bg` / `--card`	`#1d2b36` / `#5d6f7c` / `#dbe5ec` / `#f4f8fb` / `#ffffff`	Texte et surfaces
`--radius`	`16px`	Coins des cartes et encadrés

Police système (`-apple-system` , SF Pro, Segoe UI…). Taille des champs à **16 px** pour éviter le zoom automatique d'iOS. Zones tactiles ≥ 44 px. `safe-area-inset` pour l'encoche.

Catégories par défaut (`window.CATEGORIES`) : `Entrées` , `Plats` , `Desserts` , `Boulangerie` , `Bases` . Une catégorie inconnue ajoutée depuis le formulaire apparaît quand même dans les pastilles.

Recettes intégrées actuelles (26 dans `js/recettes.js`) plus les ajouts JSON :

- Entrées : `terrine-saumon` , `terrine-deux-poissons` , `souffle-fromage` , `gateau-courgettes-corinne` , `pate-campagne` (+ `navettes` dans le JSON)
- Plats : `boeuf-marrons` , `cassoulet` , `langue-boeuf-sauce-piquante` , `paella-fernande` , `quenelles-poisson-nantua` , `boudin-blanc` , `polenta`
- Desserts : `clafoutis-pommes-tatin` , `coulant-chocolat` , `crumble-aux-pommes` , `flan-patissier-quentin-lechat` , `fondant-aux-pommes` , `fondant-chocolat` , `mousse-chocolat` , `omelette-norvegienne` , `savarin-francoise` , `tarte-citron-meringuee`
- Boulangerie : `buns` , `brioche` , `pate-pizza-biga`
- Bases : `pate-brisee`

---

4. Format exact d'une recette

Une recette est un objet JavaScript (ou JSON équivalent). Le format est documenté en tête de `js/recettes.js` . Respecter les noms de champs : l'application les lit tels quels.

4.1 Champs de premier niveau

Champ	Type	Rôle
`id`	chaîne	Identifiant unique, sans espace ni accent (lettres minuscules, chiffres, tirets). Ex. `pate-pizza-biga` . Sert d'URL : `#/recette/<id>` .
`titre`	chaîne	Titre affiché.
`sousTitre`	chaîne	Sous-titre ; peut être vide `""` .
`categorie`	chaîne	Une des catégories ci-dessus, ou une autre courte.
`image`	chaîne ou `null`	Chemin relatif (`img/….jpg`). `null` si pas de photo (placeholder emoji selon la catégorie).
`infos`	objet	Textes affichés **tels quels** dans les tuiles (ils ne sont pas recalculés).
`rendement`	objet	Réglage du stepper « nombre de … ». Obligatoire.
`ingredients`	tableau	Ingrédients et intertitres de groupe.
`etapes`	tableau	Étapes et intertitres de partie.
`formats`	tableau, facultatif	Variantes (gros / mini, etc.) avec leur propre `rendement` .
`calcul`	booléen, facultatif	Si `true` , recette « calculateur » : les quantités sont **toujours** calculées, même à la valeur de base, et la mention « Fiche : » peut rappeler le texte d'origine.

`infos`

```
infos: {
  parts: "6 personnes",          // tuile Parts
  preparation: "20 min",         // tuile Préparation
  cuisson: "35 min",             // tuile Cuisson (un format peut la surcharger)
  temperature: "180 °C"          // tuile Température / repos
}
```

Si une info manque, le formulaire écrit `"Non précisé"`. Ne pas inventer un temps ou une température.

4.2 `rendement`

```
rendement: {
  valeur: 6,            // valeur de RÉFÉRENCE : les q des ingrédients correspondent à ce nombre
  depart: 6,            // facultatif : valeur affichée à l'ouverture (sinon = valeur)
  unite: "personnes",   // libellé au pluriel sous le stepper
  uniteSing: "personne",// libellé au singulier si le nombre vaut 1
  pas: 1,               // incrément des boutons + et -
  min: 1,               // plancher du stepper
  libelle: "Pour",      // facultatif : intitulé au-dessus du stepper (sinon « Pour » / « Nombre » /
« Calculateur »)
  aide: "texte…",       // facultatif : petite note sous le stepper (recette de référence, jamais un
poids qui devrait suivre le réglage)
  masseViande: 1.98     // facultatif, boudin : masse de viande affichée = stepper ÷ masseViande (en
g)
}
```

Le facteur de calcul est toujours :

```
f = st.value / rd.valeur
```

où `st.value` est la valeur courante du stepper (initialisée à `depart || valeur`).

Conséquence : pour afficher une quantité de départ différente de celle du tableur sans réécrire tous les `q`, on pose `depart` (ex. pizza : `valeur: 12`, `depart: 6` → facteur initial 0,5). C'est le mécanisme à utiliser pour « partir sur 6 pizzas » ou « partir sur 1 000 g de préparation ».

4.3 `ingredients`

Trois formes possibles dans le même tableau :

1. Ingrédient calculable

```
{
  t: "400 g de saumon frais sans peau ni arêtes",  // texte exact affiché quand f === 1 et que calcul
n'est pas true
  q: 400,        // nombre, ou [min, max] pour une fourchette
  u: "g",        // unité (g, kg, ml, cl, L, c. à café, c. à soupe, ou vide pour les pièces)
  l: "de saumon frais sans peau ni arêtes", // libellé APRÈS la quantité recalculée — SANS poids fixe
  sg: "oeuf",    // facultatif : libellé au singulier (pièces, quand la qté arrondie ≤ 1)
  pre: "Le jus de", // facultatif : texte AVANT la quantité (« Le jus de **2** citrons »)
  precis: true,  // facultatif : arrondi au dixième de gramme (levure, épices du boudin)
  ou: { q: 20, u: "g", l: "de levure fraîche" }, // facultatif : alternative « ou **20 g** de levure
fraîche »
  oeufsPour: 50,    // facultatif : ajoute « , soit environ N oeufs » (q × f / oeufsPour)
  jaunesPour: 20    // facultatif : ajoute « , environ N jaunes »
}
```

2. Intertitre de groupe

```
{ g: "Viande" }
```

Affiché en petites capitales ambre. Un `{ g: "" }` crée juste un espace visuel (utilisé sur la pizza).

3. Ligne figée (jamais recalculée)

```
{ t: "Sel et poivre" }
```

Un ingrédient **sans** `q` s'affiche toujours tel quel. S'en servir pour les mentions libres (« Beurre pour le moule », « Toute la biga », « Crème pour compenser l'évaporation » sans quantité imposée).

4.4 `etapes`

- Étape normale : `["Titre court", "Texte de l'instruction, conservé de l'ancienne fiche."]`.
- Intertitre de partie : `{ g: "La veille" }`. La numérotation **repart à 1** après un intertitre.

Ne pas résumer ni réécrire le texte des étapes quand on met à jour des quantités.

4.5 `formats` (facultatif)

Tableau d'objets. Exemple des buns :

```
formats: [
  {
    id: "gros",
    libelle: "Gros buns (environ 100 g)",
    rendement: { valeur: 8, unite: "gros buns", uniteSing: "gros bun", pas: 1, min: 1 }
  },
  {
    id: "mini",
    libelle: "Mini buns (environ 20 g)",
    rendement: { valeur: 40, unite: "mini buns", uniteSing: "mini bun", pas: 5, min: 5 },
    note: "La recette de base donne 8 gros buns…",
    // cuisson: "20 min",          // facultatif, remplace infos.cuisson
    // ingredients: [ … ]          // facultatif, liste dédiée ; sinon celle de la recette
  }
]
```

Lien profond : `#/recette/buns?format=mini&n=20` (`format` = `id` du format, `n` = valeur du stepper).

4.6 Identifiants et validité

Une recette est considérée valide (`app.js` , fonction `valide`) si :

- `id` est une chaîne ;
- `titre` est renseigné ;
- `rendement.valeur > 0` ;
- `ingredients` et `etapes` sont des tableaux.

Le formulaire construit aussi `uniteSing` (approximation : retire un `s` ou `x` final du premier mot) et `min = pas` pour les recettes ajoutées. Lors d'une **modification** d'une fiche intégrée, `conserverExtras` recopie `calcul` , `formats` , `masseViande` , `aide` , `libelle` , `depart` , `min` , ainsi que `pre` , `sg` , `ou` , `oeufsPour` , `jaunesPour` , `precis` des ingrédients reconnus par similarité de libellé.

4.7 Exemple minimal

```
{
  id: "exemple-tarte",
  titre: "Tarte aux pommes",
  sousTitre: "pâte brisée maison",
  categorie: "Desserts",
  image: "img/exemple-tarte.jpg",
  infos: { parts: "6 personnes", preparation: "20 min", cuisson: "35 min", temperature: "180 °C" },
  rendement: { valeur: 6, unite: "personnes", uniteSing: "personne", pas: 1, min: 1 },
  ingredients: [
    { t: "250 g de farine", q: 250, u: "g", l: "de farine" },
    { t: "3 pommes", q: 3, l: "pommes", sg: "pomme" },
    { t: "Sel, poivre" }
  ],
  etapes: [
    ["Préparer la pâte", "Mélanger la farine et le beurre."],
    ["Enfourner", "Cuire 35 minutes à 180 °C."]
  ]
}
```

---

5. Règles métier du propriétaire

Ces règles priment sur une lecture trop littérale d'un tableur ou d'une suggestion « pratique ».

5.1 Tableurs et anciennes fiches

- Un classeur Excel (pesées, biga, etc.) sert **uniquement** à récupérer les **quantités** et les pourcentages.
- Les **étapes**, le découpage (veille / lendemain, biga / pâte), le vocabulaire et les précautions viennent de **l'ancienne fiche**.
- Ne pas inventer une étape manquante (ex. une cuisson au four pour la pâte à pizza).

5.2 Jamais de poids fixe dans un libellé

Si un nombre dépend du réglage, il doit vivre dans `q` (et éventuellement `pre`), **pas** dans `l` ni dans un `t` sans `q`.

Contre-exemple (bug pizza) : `{ t: "Toute la biga (1 680 g)" }` — le « 1 680 g » resterait affiché même pour 6 pâtons.

Bonne pratique : `{ t: "Toute la biga" }` (ligne figée, volontairement sans poids) **ou** une quantité dans `q` avec un libellé neutre.

Autre motif correct (pâté, crème qui complète le lait) :

```
{
  t: "Crème fraîche pour compléter le lait infusé à 100 g",
  pre: "Crème fraîche pour compléter le lait infusé à",
  q: 100, u: "g", l: ""
}
```

À 2 kg de pâté, l'affichage devient « Crème fraîche pour compléter le lait infusé à **200 g** ».

5.3 Pâte à pizza (`pate-pizza-biga`)

- Deux parties bien séparées, dans les ingrédients **et** dans les étapes : **1. La biga** puis **2. La pâte**.

- Ingrédient de liaison : « Toute la biga », **sans poids**.
- **Pas d'étape de cuisson au four** dans cette fiche : on prépare des pâtons. La tuile Cuisson affiche exactement : **« Au moment de faire la pizza »**.
- Poids d'un pâton ≈ **275 g**, calculé sur farine + eau + levure. **Le sel et l'huile ne comptent pas** dans ce poids.
- Hydratation biga 44 %, hydratation globale 60 %, taux de biga 51 % de la masse farine + eau, levure 0,15 % de la farine totale, sel 2,7 %, huile 2 %.
- Quantités de référence du tableau « Quantité pate pizza sur BIGA » pour **12 pâtons** (à garder dans les `q`) :

Groupe	Ingrédient	q	Notes
Biga	Farine T00 ou T45	1166,64 g	97,22 g / pâton
Biga	Eau froide (frigo)	513,36 g	42,78 g / pâton
Biga	Levure sèche	3,09186 g	`precis: true`
Pâte	Toute la biga	—	pas de `q`
Pâte	Farine T00 ou T45	894,6 g	74,55 g / pâton
Pâte	Eau tempérée 20–22 °C	720 g	60 g / pâton
Pâte	Sel fin	55,65348 g	hors poids du pâton
Pâte	Huile d'olive	41,2248 g	hors poids du pâton

- **Affichage initial souhaité : 6 pâtons** → `valeur: 12`, `depart: 6`, `pas: 1`, `min: 1`. Les `q` restent ceux des 12 pâtons ; le facteur initial vaut 0,5.
- Photo : `img/pate-pizza-patons.jpg` (pâtons ayant fermenté, style photo réelle).

5.4 Boudin blanc (`boudin-blanc`)

- Recette `calcul: true` (calculateur, quantités toujours recalculées).
- Le stepper règle la **quantité totale de boudin à préparer**, pas la masse de viande.
- **Base d'affichage : 1 000 g de préparation** (`depart: 1000`).
- Les `q` correspondent à **1 kg de viande**, soit **1 980 g** de préparation (`valeur: 1980`). D'où `masseViande: 1.98` : la masse de viande affichée sous le stepper est `round(st.value / 1.98)` grammes. Pour 1 000 g de boudin ≈ **505 g** de viande.
- `pas: 250`, `min: 500`, `libelle: "Quantité de boudin à préparer"`, `unite` / `uniteSing` : `"g de boudin au total"`.
- Sous le stepper : une seule ligne d'information, du type « Masse de viande : **505 g** (les calculs se font sur cette masse) ». Pas d'exposé pédagogique supplémentaire.
- Quantités pour 1 kg de viande / 1 980 g de préparation :

Groupe	Ingrédient	q
Viande	Maigre 75 %	750 g
Viande	Gras 25 %	250 g
Farce	Mie de pain 23 %	230 g
Farce	Lait demi-écrémé aromatisé 50 %	500 g
Farce	Crème fraîche pour l'évaporation	*sans* `q`
Farce	Oignon cuit au beurre, singé	120 g
Farce	Maïzena 4 %	40 g
Farce	Œufs entiers 25 %	250 g (`oeufsPour: 50`)
Épicerie (sur la masse totale)	Sel 1,5 %	29,7 g
Épicerie	Poivre blanc 0,25 %	4,95 g (`precis`)
Épicerie	Quatre-épices 0,1 %	1,98 g (`precis`)
Épicerie	Muscade 0,1 %	1,98 g (`precis`)
Épicerie	Porto / cognac / madère 0,25 %	**4,95 cl** (unité `cl` , pas des grammes)
Épicerie	Truffe 3 % (appellation truffée)	59,4 g
—	Boyau (15 cm par boudin de 125 g)	**2,38 m**

- Le boyau se recalcule avec `f` comme les autres `q` : pour 1 kg de préparation, environ 1,20 m.
- Les mentions du type « 16 boudins de 125 g pour 1 kg de viande » dans `infos.parts` **ne se recalculent pas** (texte figé des tuiles). Ne pas promettre le contraire.
- Conserver les étapes de l'ancienne fiche (infusion du lait, singer, panade, hachage, embossage, pochage 75–85 °C, refroidissement).

5.5 Pâté de campagne (`pate-campagne`)

- Base **1 kg** (`rendement.valeur: 1` , unité « kg de pâté », `pas` et `min` à 0,5).
- Sel inchangé : 12 g de sel fin + 10 g de sel nitrité.
- Sucre : **2 g**, quantité estimée d'après des recettes de référence (le mentionner dans le `t` / `l` si on garde cette transparence).
- Vin blanc (250 g) et cognac (10 g) : **pour la marinade**, pas pour un mouillage de la farce.
- Lait : **100 g** à aromatiser ; une partie s'évapore ; **compléter avec de la crème jusqu'à 100 g** (motif `pre` ci-dessus).
- Photo rustique, viande peu hachée : `img/pate-campagne-rustique.jpg` .
- Étapes groupées : « La veille », « Le lendemain », « Cuisson ».

5.6 Photos et langue

- Préférer des visuels **réalistes, type photographie**, pas des illustrations.
- Le pâté doit rester **rustique**.
- Toute l'interface, les libellés, les messages d'erreur et les conversations avec le propriétaire sont en **français**.

6. Calculateur de quantités

Implémenté dans `js/app.js`.

6.1 Facteur et état

Pour la fiche `r` et l'état `st = recState[r.id]` :

```
fmt = format courant (ou la recette elle-même)
rd  = fmt.rendement
f   = st.value / rd.valeur
```

`st` contient `{ format, value, done }` où `done` mémorise les ingrédients cochés. À la première ouverture :

```
value = rd.depart || rd.valeur
```

Changer de format réinitialise `value` à `depart || valeur` du nouveau format et vide les coches.

6.2 Affichage d'un ingrédient (`ingredientHTML`)

1. Pas de `q` → afficher `t` tel quel.
2. Si `f === 1` **et** que la recette n'est pas `calcul` **et** que `t` existe → afficher `t` tel quel (texte d'origine de la fiche).
3. Sinon, calculer `q × f` (ou chaque borne d'une fourchette), arrondir, puis composer :

`[pre] **quantité [unité]** libellé`

Le libellé est `sg` s'il n'y a pas d'unité et que la quantité arrondie est ≤ 1, sinon `l`. 4. Ajouter l'alternative `ou`, puis `oeufsPour` / `jaunesPour`. 5. Si `r.calcul` et `t` : ajouter une ligne `<span class="ing-rule">Fiche : …</span>` avec le texte d'origine. Utile sur le boudin pour rappeler le pourcentage de la fiche.

6.3 Arrondis (`roundQty`)

Cas	Règle
`precis: true`	`max(0,1 ; arrondi à 0,1)`
Recette `calcul` et unité `g` ou `ml`	Si < 10 g : au demi-gramme ; sinon à l'unité
`g` / `ml` (recette normale)	< 2 → au moins 0,5 au demi ; < 10 → au demi ; < 100 → unité ; < 500 → multiple de 5 ; sinon multiple de 10
`kg` / `L`	Au 0,05 près (plancher 0,05)
`cl`	< 10 → au demi (plancher 0,5) ; sinon à l'unité
Pièces, cuillères, pincées, sachets…	Au demi, plancher 0,5

`fmtNum` affiche une virgule décimale française, une espace fine comme séparateur de milliers, et la fraction `½` pour les unités « comptées ».

6.4 Interface du réglage

- Titre du bloc : « Format et quantité » s'il y a des `formats`, « Calculateur » si `calcul`, sinon « Quantité ».

- Stepper ambre, boutons − / +. Le bouton − est désactivé à `min`.
- Si `masseViande` est défini : ligne « Masse de viande : **N g** (les calculs se font sur cette masse) ».
- Si `aide` : afficher le texte (ex. détail farine/eau par pâton, « 1 flan = 8 personnes »).
- Si `fmt.note` : idem (note des mini-buns).
- **Pas de bandeau jaune** du type « Quantités recalculées (× 1,5). Les quantités citées dans les étapes correspondent à la recette d'origine. » Ce bandeau a été retiré à la demande du propriétaire. Quand `st.value` diffère de `depart || valeur`, afficher uniquement le bouton **« Revenir à la recette d'origine »**, qui remet `st.value` à `depart || valeur`.
- Les textes des **étapes** et des **tuiles** `infos` ne sont jamais multipliés. Si une étape cite « les 100 g de sucre », cela reste le libellé de la fiche d'origine.

6.5 Accueil et recherche

- Recherche : normalisation minuscule, `œ` → `oe`, suppression des accents ; tous les mots doivent se retrouver dans titre + sous-titre + catégorie + textes d'ingrédients.
- Pastilles de catégories présentes dans le livre, plus « Toutes ».
- Compteur « N recette(s) ». Cartes avec photo 16:9, catégorie, titre, sous-titre, temps de préparation et de cuisson.
- Bouton **「+ Ajouter une recette」** sous la barre de recherche → `#/ajouter`.

---

7. Ajout, modification et suppression depuis le téléphone

Tout est dans `js/ajout.js`. Constantes :

```
DEPOT            = "nicobuq8-hue/livre-recettes"
BRANCHE          = "main"
FICHIER          = "data/recettes-ajoutees.json"
FICHIER_MASQUEES = "data/recettes-masquees.json"
API              = https://api.github.com/repos/<DEPOT>/contents/
CLE_JETON        = "livre-recettes-jeton-github"      // localStorage
CLE_ATTENTE      = "livre-recettes-attente"           // localStorage, 10 minutes
```

7.1 Jeton GitHub

- Type : **fine-grained personal access token** (`github_pat_…`).
- Création : https://github.com/settings/personal-access-tokens/new
- Nom suggéré : « Livre de recettes » ; durée au choix (ex. 1 an).
- *Repository access* : **Only select repositories** → `nicobuq8-hue/livre-recettes` (pas « Public repositories », sinon les permissions restent en lecture seule).
- Permission **Contents : Read and write**. Le menu « Read and write » n'apparaît qu'**après** avoir coché Contents, puis validé.
- Le jeton est saisi dans « Réglages GitHub » du formulaire, enregistré uniquement dans `localStorage`. **Ne jamais le committer, ni l'écrire dans un fichier du dépôt, ni le journaliser.**
- Boutons : Enregistrer, Vérifier (GET du dépôt, contrôle de `permissions.push`), Effacer (avec confirmation).

7.2 Fusion au chargement (`chargerAjouts` dans `app.js`)

Ordre :

1. Partir de `window.RECETTES` (fichier JS).
2. Télécharger `data/recettes-ajoutees.json` et `data/recettes-masquees.json` en `cache: "no-store"`.
3. Construire l'ensemble des ids masqués = JSON masquées ∪ suppressions encore en attente (`localStorage`).
4. Pour chaque recette du JSON d'ajouts : si elle est `valide` et non masquée, l'**intégrer** (même `id` qu'une recette intégrée → **surcharge**, le JS n'est pas réécrit).
5. Intégrer aussi les ajouts encore en attente de publication Pages (10 min).
6. Retirer de la liste affichée tous les ids masqués.

Champs internes après intégration : `_ajout = true` ; `_apercu` éventuellement (data-URL de la photo juste envoyée, le temps que Pages publie le JPEG).

7.3 Ajout

1. L'utilisateur remplit le formulaire (titre *, catégorie, nombre de base *, unité, pas, infos, ingrédients, étapes, photo facultative).
2. `id` généré par slug du titre (normalisé, 60 caractères max) ; suffixe `-2` , `-3` … si collision, y compris avec une recette masquée.
3. Photo : redimensionnée (plus grand côté ≤ 1 200 px), JPEG qualité 0,8, envoyée en `img/ajout-<id>.jpg` .
4. La recette est ajoutée au tableau de `data/recettes-ajoutees.json` (commit `Ajout recette : <titre>`).
5. Affichage immédiat sur le téléphone via `integrer` + file d'attente ; sur les autres appareils, délai Pages d'**1 à 2 minutes**.

Si l'utilisateur quitte la page avant d'envoyer, la saisie est perdue (pas de brouillon).

7.4 Modification

- Toutes les fiches ont un bouton ✏️ **Modifier** → `#/modifier/<id>` .
- Modifier une recette **intégrée** écrit une surcharge de même `id` dans `recettes-ajoutees.json` (commit `Modification recette : <titre>`), sans toucher à `js/recettes.js` .
- `conserverExtras` préserve les champs avancés absents du formulaire (voir 4.6).
- Pour intégrer définitivement une surcharge dans le JS : copier l'objet dans `js/recettes.js` , le retirer du JSON, et le retirer de `recettes-masquees.json` s'il y figure.

7.5 Suppression

- Toutes les fiches ont 🗑️ **Supprimer**.
- Confirmation obligatoire : `Supprimer « <titre> » du livre ? Cette action est définitive.`
- Recette **uniquement présente** dans le JSON : elle est retirée du tableau ; la photo `img/ajout-<id>.jpg` est effacée si le chemin correspond.
- Recette **intégrée** : son `id` est poussé dans `data/recettes-masquees.json` (commit `Masquage recette : <titre>`). Le JS n'est pas réécrit depuis le téléphone. Une surcharge éventuelle est aussi retirée du JSON, pour que l'originale ne réapparaisse pas.
- Sans jeton, l'application renvoie vers le formulaire pour le configurer.

En cas de conflit SHA (HTTP 409), une relecture unique est tentée. Les erreurs 401 / 403 / 404 / 5xx ont des messages en français explicites.

7.6 Routes

Hash	Écran
`#/`	Accueil
`#/recette/<id>`	Fiche
`#/recette/<id>?format=…&n=…`	Fiche avec format et quantité imposés
`#/ajouter`	Formulaire de création
`#/modifier/<id>`	Formulaire d'édition prérempli

---

8. Import depuis une IA

Bloc « ✨ Importer depuis une IA » en haut du formulaire (`#/ajouter` et `#/modifier/…`).

8.1 Parcours utilisateur

1. 📋 **Copier la consigne pour l'IA** copie le texte `CONSIGNE_IA` (constante dans `js/ajout.js`). Si le presse-papiers est bloqué, « Voir la consigne » permet une copie manuelle.
2. L'utilisateur colle la consigne dans ChatGPT, Gemini, Grok, Claude, etc., et joint la photo (magazine, manuscrit) ou le texte.
3. Il colle **toute** la réponse dans « Colle ici la réponse de l'IA » puis appuie sur **Remplir le formulaire**.
4. Il **relit**, complète (photo du plat notamment), puis envoie.

Rien n'est envoyé vers GitHub automatiquement. L'import ne fait que remplir les champs.

8.2 Format attendu de la réponse

Un unique objet JSON, sans texte autour (des fences ```` ```json ```` sont toutefois acceptées). Champs simples :

```
{
  "titre": "Tarte aux pommes",
  "sousTitre": "pâte brisée maison",
  "categorie": "Desserts",
  "infos": {
    "parts": "6 personnes",
    "preparation": "20 min",
    "cuisson": "35 min",
    "temperature": "180 °C"
  },
  "rendement": { "valeur": 6, "unite": "personnes" },
  "ingredients": [
    { "groupe": "Pâte" },
    { "quantite": 250, "unite": "g", "libelle": "de farine" },
    { "quantite": 2, "unite": "pièce", "libelle": "oeufs" },
    { "quantite": null, "unite": "", "libelle": "Sel, poivre" }
  ],
  "etapes": [
    { "titre": "Préparer la pâte", "texte": "Mélanger la farine et le beurre…" }
  ]
}
```

Règles dictées à l'IA tierce : tout en français ; `quantite` en nombre avec un **point** décimal ; fourchette → première valeur ; ne rien inventer ; garder le texte d'origine des étapes.

8.3 Analyse tolérante (`depuisIA`)

- Extraction du premier bloc `{ … }` équilibré, y compris si la réponse est coupée.
- Suppression des fences Markdown ; remplacement des guillemets typographiques.
- Seconde tentative après suppression des virgules traînantes.
- Clés reconnues malgré accents, majuscules, synonymes français/anglais (`title` , `quantity` , `steps` , `q` / `l` / `t` du format interne, etc.).
- Quantités comprises sous les formes `1,5` , `1/2` , `1 1/2` , `½` , `2-3` , `[2, 3]` .
- Unités normalisées (`gr` → `g` , `cs` → `c. à soupe` , `litre` → `L` , `pièce` / `pcs` → pas d'unité, etc.).
- Ingrédient chaîne brute, étape tableau `[titre, texte]` , ou objet `{ g: "…" }` acceptés.
- Après remplissage, un message liste ce qui manque encore (titre, nombre, ingrédients, étapes). Le champ réponse est vidé.

La consigne complète à recopier en cas de recréation du fichier se trouve dans `js/ajout.js` (`var CONSIGNE_IA = [ … ]`).

---

9. Déploiement

9.1 Publication normale

1. Modifier les fichiers du dépôt (recette, style, script, photo…).
2. Lancer `python3 build-sw.py` dès qu'un fichier de `css/` , `js/` , `icons/` , `img/` ou `data/` change. Cela réécrit `sw.js` (nouvelle version de cache). Inutile pour le seul dossier `docs/` .
3. Contrôler la syntaxe : `node --check js/app.js && node --check js/ajout.js && node --check js/recettes.js` .
4. Tester en local : `python3 -m http.server 8000` puis http://localhost:8000.
5. Commit sur `main` avec un message clair en français (ex. `Boudin blanc : quantités du classeur Pesee boudin saucisses`).
6. `git push origin main` . GitHub Pages republie en 1 à 2 minutes.

9.2 Incidents fréquents

- **Push en erreur 500** (GitHub instable) : **réessayer** le push, sans changer l'historique.
- **Pages ne se reconstruit pas** : déclencher un build manuel `gh api -X POST repos/nicobuq8-hue/livre-recettes/pages/builds`
- Le connecteur GitHub de l'éditeur peut rester bloqué : se servir de `gh auth login` (device flow) sur le compte `nicobuq8-hue` .
- Sur iPhone, après une publication : **fermer puis rouvrir** le livre (le SW va chercher le réseau, mais un onglet déjà ouvert peut garder d'anciens scripts en mémoire).

9.3 Installation sur iPhone

Le site doit être en HTTPS (c'est le cas sur GitHub Pages). Dans Safari : bouton **Partager** → **Sur l'écran d'accueil**. L'icône est la photographie de l'ancien livre de recettes (`icons/`).

9.4 Ce qu'il ne faut pas publier

- Aucun jeton, secret, fichier `.env`, ni capture montrant un `github_pat_`.
- Ne pas servir ce guide comme page d'accueil : Pages est configuré sur la **racine** du dépôt, pas sur `/docs`.

---

10. Comment une IA doit travailler

1. **Lire** ce guide **et** le dépôt (`LISEZMOI.md`, en-tête de `js/recettes.js`, puis le code concerné). Ne pas se fier à une ancienne conversation si le fichier dit autre chose — sauf pour les règles métier du § 5, qui priment.
2. **Parler français** à l'utilisateur, phrases claires, tutoiement ou formulaires déjà employés dans l'app (« tu », phrases courtes). Annoncer le lien de la fiche mise à jour : `https://nicobuq8-hue.github.io/livre-recettes/#/recette/<id>`
3. **Choisir le bon fichier**
   - Nouvelle recette « officielle » ou correction de fond d'une fiche intégrée : `js/recettes.js`.
   - Ajout / modification faite comme depuis le téléphone, ou surcharge temporaire : `data/recettes-ajoutees.json`.
   - Masquage : `data/recettes-masquees.json`.
   Ne pas dupliquer la même fiche dans le JS **et** le JSON (le JSON gagnerait au chargement, ce qui prête à confusion).
4. **Quantités vs étapes** : reprendre les chiffres du tableur ; conserver le texte des étapes existantes.
5. **Libellés** : aucun poids fixe qui devrait varier. Vérifier particulièrement « Toute la biga », les crèmes « jusqu'à N g », les pâtons.
6. **Respecter le CSS et le mobile-first** : pas de framework, pas de largeur fixe type desktop, champs à 16 px, contrastes sarcelle / ambre déjà en place.
7. **Photos** : réalistes ; nommer clairement le fichier ; placer dans `img/` ; mettre à jour le champ `image`. Pour un envoi formulaire : `img/ajout-<id>.jpg`.
8. **Après chaque changement de ressources** : `python3 build-sw.py`, puis `node --check` sur les JS touchés.
9. **Commit / push** sur `main` ; réessayer si GitHub renvoie 500 ; déclencher un build Pages si rien ne se passe. Prévenir l'utilisateur de fermer et rouvrir le livre.
10. **Ne jamais committer de jeton**, ni l'écrire dans le code, les commentaires, les captures ou ce guide.
11. **Ne pas casser le calculateur** : `f = st.value / rd.valeur` ; `depart` pour l'affichage initial ; pas de bandeau jaune de multiplication ; lignes « Fiche : » seulement si `calcul: true`.
12. Si une demande mélange une correction de fiche **et** une fonctionnalité risquée, traiter d'abord la fiche, clairement.

---

11. Liens et chemins utiles

Ressource	URL ou chemin
Livre en ligne	https://nicobuq8-hue.github.io/livre-recettes/
Accueil / ajout	https://nicobuq8-hue.github.io/livre-recettes/#/ajouter
Boudin blanc	https://nicobuq8-hue.github.io/livre-recettes/#/recette/boudin-blanc
Pâte à pizza	https://nicobuq8-hue.github.io/livre-recettes/#/recette/pate-pizza-biga
Pâté de campagne	https://nicobuq8-hue.github.io/livre-recettes/#/recette/pate-campagne
Dépôt GitHub	https://github.com/nicobuq8-hue/livre-recettes
Fichier des recettes	`js/recettes.js`
Application	`js/app.js`
Formulaire / GitHub / IA	`js/ajout.js`
Styles	`css/style.css`
Ajouts / surcharges	`data/recettes-ajoutees.json`
Masquages	`data/recettes-masquees.json`
Générateur du SW	`build-sw.py`
Nouveau jeton	https://github.com/settings/personal-access-tokens/new
Ce guide (Markdown)	`docs/GUIDE-RECREATION-IA.md`
Ce guide (PDF)	`docs/Guide-recreation-livre-recettes-IA.pdf`

12. Annexes

12.1 Calcul du pâton de pizza (rappel)

```
farine_totale = 1166,64 + 894,6 = 2061,24 g
eau_totale    = 513,36 + 720    = 1233,36 g
levure        = 3,09186 g
sel           = 2,7 % × 2061,24 = 55,65348 g   ← exclu du pâton
huile         = 2,0 % × 2061,24 = 41,2248 g    ← exclu du pâton

masse_12_pâtons = 2061,24 + 1233,36 + 3,09186 = 3297,69186 g
pâton           = 3297,69186 / 12 ≈ 274,8 g ≈ 275 g
```

12.2 Calculateur boudin : relations

```
masse_viande_affichée_g = round( stepper / 1,98 )
f                       = stepper / 1980
q_affichée              = arrondi( q_référence × f )

boyau_référence = 15 cm × (1980 g / 125 g) = 237,6 cm = 2,38 m
```

Les pourcentages de la farce (23 %, 50 %, 12 %, 4 %, 25 %) s'appliquent à la **masse de viande**. Les pourcentages d'épicerie (1,5 %, 0,25 %, 0,1 %, 3 %) s'appliquent à la **masse totale de préparation** (1 980 g pour 1 kg de viande).

12.3 Checklist avant d'ajouter une recette dans `recettes.js`

- [ ] `id` unique, slug sans accent
- [ ] `categorie` cohérente
- [ ] `infos` complets ; « Non précisé » plutôt qu'une invention
- [ ] `rendement.valeur` = la base à laquelle correspondent les `q`
- [ ] `depart` si l'affichage initial doit différer de cette base
- [ ] Chaque `l` sans poids fixe ; `t` fidèle à la fiche d'origine
- [ ] Groupes `g` pour séparer les parties (biga / pâte, viande / farce, veille…)
- [ ] Étapes reprises de l'ancienne fiche, éventuellement avec `{ g: "…" }`
- [ ] Photo réaliste dans `img/`, chemin renseigné
- [ ] `python3 build-sw.py` + `node --check`

12.4 Ce que le formulaire ne permet pas (à faire dans le JS)

Le formulaire téléphone gère un calcul simple. Les règles particulières restent plus sûres dans `js/recettes.js` :

- `calcul`, `formats`, `masseViande`, `depart`, `aide`, `libelle` personnalisé ;
- `precis`, `pre`, `sg`, `ou`, `oeufsPour`, `jaunesPour` ;
- fourchettes `[min, max]`.

`conserverExtras` les préserve lors d'une modification ultérieure depuis le téléphone, s'il reconnaît l'ingrédient.

Fin du guide. En cas de doute, ouvrir le fichier source correspondant : c'est lui qui fait foi pour le comportement actuel, et le § 5 qui fait foi pour l'intention du propriétaire.